

Automated Testing Report

1) Automation tool

- Playwright

2) Automated Testing Report

a) Use Case 17 — Dashboard Concept Graph

Testcase 1:

- **ID:** TC-DB-020
- **Test case name:** Search concepts by name
- **Test result:** Pass
- **Test script:**

```
test.describe('TC-DB-020: Tìm kiếm node theo tên trên đồ thị', () => {
  test('Làm mờ chính xác node không khớp và xóa tìm kiếm vẫn giữ
panel đang mở', async ({
    page,
  }) => {
    const seed = await seedDashboardData(prisma, 'tc_db_020', {
      emptyQueue: true,
      seedActivity: false,
    });

    try {
      const [p1] = seed.plans;
      if (!p1) throw new Error('Seed TC-DB-020 thiếu P1.');
```

// 1. Chuẩn bị năm node có tên và mastery riêng để kiểm tra một, nhiều và không có kết quả.

```
const initialConcepts = await prisma.concept.findMany({
  where: { planId: p1.id },
  orderBy: { name: 'asc' },
  select: { id: true },
});
const [c1, c2, c3] = initialConcepts;
if (!c1 || !c2 || !c3) throw new Error('Seed TC-DB-020 thiếu concept
mặc định.');
```

```
await Promise.all([
  prisma.concept.update({
```

```

    where: { id: c1.id },
    data: { name: 'Cây AVL', masteryScore: 0.8 },
  }),
  prisma.concept.update({
    where: { id: c2.id },
    data: { name: 'Cây nhị phân', masteryScore: 0.6 },
  }),
  prisma.concept.update({
    where: { id: c3.id },
    data: { name: 'Đồ thị có hướng', masteryScore: 0.4 },
  }),
]);
await prisma.concept.createMany({
  data: [
    { planId: p1.id, name: 'Mảng động', masteryScore: null },
    { planId: p1.id, name: 'Ngăn xếp', masteryScore: 0.8 },
  ],
});

```

// 2. Đăng nhập và mở graph đầy đủ của P1 ở chế độ chỉ xem.

```

await loginViaUi(page, seed.user.email);
await page.goto(`/plan/${p1.id}`);
const graphNodes = page.locator('.react-flow__node');
await expect(graphNodes).toHaveCount(5);
const searchInput = page.getByLabel('Tìm khái niệm', { exact: true
});
const avlNode = graphNodes.filter({ hasText: 'Cây AVL' });
const binaryNode = graphNodes.filter({ hasText: 'Cây nhị phân' });
const directedGraphNode = graphNodes.filter({ hasText: 'Đồ thị có
hướng' });
const arrayNode = graphNodes.filter({ hasText: 'Mảng động' });
const stackNode = graphNodes.filter({ hasText: 'Ngăn xếp' });

```

// 3. Tìm không phân biệt hoa/thường một node và xác nhận bốn node còn lại bị làm mờ.

```

await searchInput.fill('cây avl');
await expect(avlNode).not.toHaveClass(/is-dimmed/);
await expect(binaryNode).toHaveClass(/is-dimmed/);
await expect(directedGraphNode).toHaveClass(/is-dimmed/);
await expect(arrayNode).toHaveClass(/is-dimmed/);
await expect(stackNode).toHaveClass(/is-dimmed/);
await expect(page.locator('.react-flow__edge')).toHaveCount(1);

```

// 4. Tìm theo prefix để chỉ hai node chứa "Cây" còn rõ, không thêm node không liên quan.

```
await searchInput.fill('CÂY');
await expect(avlNode).not.toHaveClass(/is-dimmed/);
await expect(binaryNode).not.toHaveClass(/is-dimmed/);
await expect(directedGraphNode).toHaveClass(/is-dimmed/);
await expect(arrayNode).toHaveClass(/is-dimmed/);
await expect(stackNode).toHaveClass(/is-dimmed/);
```

// 5. Mở panel của một node trước khi chuyển sang trạng thái không có kết quả.

```
const detailResponse = page.waitForResponse(
  (response) =>
    new
URL(response.url()).pathname.includes(`/api/v1/plans/${p1.id}/concepts/`) &&
    response.request().method() === 'GET' &&
    response.status() === 200
);
await avlNode.click();
await detailResponse;
const avlPanel = page.locator('main aside').filter({
  has: page.getByRole('heading', { name: 'Cây AVL', exact: true }),
});
await expect(avlPanel).toBeVisible();
```

// 6. Tìm tên không tồn tại: toàn bộ node mờ và thông báo có lỗi xóa bộ lọc xuất hiện.

```
await searchInput.fill('không-tồn-tại');
await expect(page.getByText('Không có khái niệm nào khớp', {
exact: true })).toBeVisible();
await expect(page.getByRole('button', { name: 'Xóa bộ lọc', exact:
true })).toBeVisible();
await expect(avlNode).toHaveClass(/is-dimmed/);
await expect(binaryNode).toHaveClass(/is-dimmed/);
await expect(directedGraphNode).toHaveClass(/is-dimmed/);
await expect(arrayNode).toHaveClass(/is-dimmed/);
await expect(stackNode).toHaveClass(/is-dimmed/);
```

// 7. Xóa từ khóa và xác nhận toàn graph cùng panel đang mở được khôi phục nguyên vẹn.

```
await searchInput.fill("");
await expect(page.getByText('Không có khái niệm nào khớp', {
exact: true })).toHaveCount(0);
await expect(avlNode).not.toHaveClass(/is-dimmed/);
await expect(binaryNode).not.toHaveClass(/is-dimmed/);
await expect(directedGraphNode).not.toHaveClass(/is-dimmed/);
```

```

    await expect(arrayNode).not.toHaveClass(/is-dimmed/);
    await expect(stackNode).not.toHaveClass(/is-dimmed/);
    await expect(avlPanel).toBeVisible();
  } finally {
    // 8. Dọn toàn bộ dữ liệu seed qua Student để không ảnh hưởng
    // lần chạy sau.
    await prisma.user.delete({ where: { id: seed.user.id } });
  }
});
});

```

Testcase 2

- **ID:** TC-DB-021
- **Test case name:** Display the correct mastery tooltip when hovering over a concept
- **Test result:** Pass
- **Test script:**

```

test.describe('TC-DB-021: Hover tooltip hiển thị tên và mastery_score của node', () => {
  test('Tooltip cập nhật đúng cho node chưa kiểm tra, đang học và vững', async ({ page }) => {
    const seed = await seedDashboardData(prisma, 'tc_db_021', {
      emptyQueue: true,
      seedActivity: false,
    });

    try {
      const [p1] = seed.plans;
      if (!p1) throw new Error('Seed TC-DB-021 thiếu P1.');

```

// 1. Đặt ba node đại diện cho score null, 0.60 và 0.80 cùng trạng thái ngày kiểm tra rõ ràng.

```

      const concepts = await prisma.concept.findMany({
        where: { planId: p1.id },
        orderBy: { name: 'asc' },
        select: { id: true },
      });
      const [c1, c4, c6] = concepts;
      if (!c1 || !c4 || !c6) throw new Error('Seed TC-DB-021 thiếu concept mặc định.');
```

```

      await Promise.all([
        prisma.concept.update({

```

```

    where: { id: c1.id },
    data: { name: 'C1', masteryScore: null, lastTestedAt: null },
  }),
  prisma.concept.update({
    where: { id: c4.id },
    data: { name: 'C4', masteryScore: 0.6, lastTestedAt: new Date()
  },
  }),
  prisma.concept.update({
    where: { id: c6.id },
    data: { name: 'C6', masteryScore: 0.8, lastTestedAt: new Date()
  },
  }),
]);

```

// 2. Đăng nhập và mở graph chỉ xem, nơi hover không được mở panel chi tiết.

```

await loginViaUi(page, seed.user.email);
await page.goto(`/plan/${p1.id}`);
const c1Node = page.locator('.react-flow__node').filter({ hasText:
/^C1—$/ });
const c4Node = page.locator('.react-flow__node').filter({ hasText:
/^C40\.60$/ });
const c6Node = page.locator('.react-flow__node').filter({ hasText:
/^C60\.80$/ });
await expect(c1Node).toBeVisible();
await expect(c4Node).toBeVisible();
await expect(c6Node).toBeVisible();
await expect(page.locator('main aside')).toHaveCount(0);

```

// 3. Hover C1: tooltip dùng dấu — và "chưa kiểm tra", không hiển thị null hoặc 0 giả.

```

await c1Node.hover();
const c1Tooltip = page.locator('.react-flow__node-toolbar').filter({
hasText: /^C1/ });
await expect(c1Tooltip).toBeVisible();
await expect(c1Tooltip).toContainText('C1');
await expect(c1Tooltip).toContainText('—');
await expect(c1Tooltip).toContainText('chưa kiểm tra');
await expect(c1Tooltip).not.toContainText('null');
await
expect(c1Node.locator('[data-slot="concept-node"]')).toHaveAttribute(
  'data-band',
  'untested'
);

```

```

    await expect(page.locator('main aside')).toHaveCount(0);

    // 4. Hover C4 để tooltip đổi sang đúng tên, score hai chữ số và
    dài Đang học.
    await c4Node.hover();
    const c4Tooltip = page.locator('.react-flow__node-toolbar').filter({
    hasText: /^C4/ });
    await expect(c4Tooltip).toBeVisible();
    await expect(c4Tooltip).toContainText('C4');
    await expect(c4Tooltip).toContainText('0.60');
    await expect(c4Tooltip).toContainText('kiểm tra lần cuối');
    await
    expect(c4Node.locator('[data-slot="concept-node"]')).toHaveAttribute(
        'data-band',
        'learning'
    );

    // 5. Hover liên tiếp C6 để xác nhận tooltip không giữ dữ liệu cũ và
    dài chuyển sang Vững.
    await c6Node.hover();
    const c6Tooltip = page.locator('.react-flow__node-toolbar').filter({
    hasText: /^C6/ });
    await expect(c6Tooltip).toBeVisible();
    await expect(c6Tooltip).toContainText('C6');
    await expect(c6Tooltip).toContainText('0.80');
    await expect(c6Tooltip).not.toContainText('C4');
    await
    expect(c6Node.locator('[data-slot="concept-node"]')).toHaveAttribute(
        'data-band',
        'strong'
    );

    // 6. Rê ra vùng canvas trống để tooltip ẩn mà không kích hoạt
    panel của C6.
    const pane = page.locator('.react-flow__pane').first();
    await pane.hover({ position: { x: 10, y: 10 } });
    await expect(c6Tooltip).not.toBeVisible();
    await expect(page.locator('main aside')).toHaveCount(0);
    } finally {
    // 7. Dọn toàn bộ dữ liệu của Student độc lập sau từng lần chạy.
    await prisma.user.delete({ where: { id: seed.user.id } });
    }
    });
    });

```

b) Use Case 09 — Configure Focus Session (FS-02)

Testcase 1:

- **ID:** TC-FS-027
- **Test case name:** Clamp invalid Pomodoro durations to the allowed boundaries
- **Test result:** Pass
- **Test script:**

```
test('TC-FS-027: kẹp các trường Pomodoro về đúng biên', async ({
  page }) => {
  const seed = await seedFocusPlan(prisma, 'tc_fs_027');
  try {
    // 1. Mở panel trước khi bắt đầu và thử các giá trị ngoài biên.
    await loginViaUi(page, seed.user.email);
    await page.goto('/focus');
    await page.getByRole('button', { name: 'Đổi độ dài lượt', exact:
true }).click();
    const dialog = page.getByRole('dialog', { name: 'Cấu hình
Pomodoro', exact: true });
    const fields = dialog.locator('input[type="number"]');
    await fields.nth(0).fill('0');
    await expect(fields.nth(0)).toHaveValue('1');
    await fields.nth(0).fill('-5');
    await expect(fields.nth(0)).toHaveValue('1');
    await fields.nth(0).fill('200');
    await expect(fields.nth(0)).toHaveValue('120');
    await dialog.getByRole('button', { name: 'Áp dụng', exact: true
}).click();
    await expect(page.getByText('120 phút', { exact: false
})).toBeVisible();
  } finally {
    await prisma.user.delete({ where: { id: seed.user.id } });
  }
});
```

Testcase 2

- **ID:** TC-FS-028
- **Test case name:** Compare Strict Mode ON/OFF when leaving the browser tab
- **Test result:** Pass

- **Test script:**

```
test('TC-FS-028: strict ON tạm dừng timer khi rời tab, OFF tiếp tục timer', async ({
  page,
  context,
}) => {
  const seed = await seedFocusPlan(prisma, 'tc_fs_028');
  const otherPage = await context.newPage();
  try {
    // 1. Dừng clock ảo và tab phụ để phát visibilitychange ổn định
    trong headless browser.
    await loginViaUi(page, seed.user.email);
    await page.clock.install();
    await page.clock.pauseAt(new Date(Date.now() + 60_000));
    await otherPage.goto('about:blank');
    await page.bringToFront();
    await startSession(page, true);
    const timer = page.getByRole('timer');
    const beforeStrict = await readClockSeconds(timer);
    await otherPage.bringToFront();
    await dispatchVisibility(page, true);
    await page.clock.runFor(5_000);
    await page.bringToFront();
    await dispatchVisibility(page, false);
    // 2. Strict ON phải hiện cảnh báo và giữ nguyên số giây còn lại.
    await expect(
      page.getByRole('heading', { name: 'Đồng hồ tập trung đã dừng',
exact: true })
    ).toBeVisible();
    await expect(page.getByRole('button', { name: 'Tiếp tục', exact:
true })).toBeVisible();
    await page.getByRole('button', { name: 'Tiếp tục', exact: true
}).click();
    await expect(page.getByRole('button', { name: 'Tạm dừng', exact:
true })).toBeVisible();
    const afterStrict = await readClockSeconds(timer);
    expect(afterStrict).toBeGreaterThanOrEqual(beforeStrict - 1);
    expect(afterStrict).toBeLessThanOrEqual(beforeStrict);

    // Gọi lại cảnh báo để chọn nhánh tắt strict ngay trong phiên.
    await otherPage.bringToFront();
    await dispatchVisibility(page, true);
    await page.clock.runFor(1_000);
    await page.bringToFront();
    await dispatchVisibility(page, false);
```



```
    await expect(
      page.getByRole('button', { name: 'Tắt chế độ nghiêm ngặt',
exact: true })
    ).toBeVisible();
    await page.getByRole('button', { name: 'Tắt chế độ nghiêm ngặt',
exact: true }).click();
    await page.getByRole('button', { name: 'Tạm dừng', exact: true
}).waitFor();
    const beforeLoose = await readClockSeconds(timer);
    await otherPage.bringToFront();
    await dispatchVisibility(page, true);
    await page.clock.runFor(5_000);
    await page.bringToFront();
    await dispatchVisibility(page, false);
    // 3. Tắt strict từ cảnh báo; lần rồi tiếp theo tiếp tục trừ timer.
    expect(await
readClockSeconds(timer)).toBeLessThan(beforeLoose);
  } finally {
    await otherPage.close().catch(() => undefined);
    await prisma.user.delete({ where: { id: seed.user.id } });
  }
});
```